

Lab 1: Exploring Particle Swarm Optimisation (PSO) Fundamentals

This laboratory session introduces students to **Particle Swarm Optimisation (PSO)** as a core **nature-inspired optimisation technique** within Computational Intelligence. Building on the concepts introduced in Lecture 1, students will implement and experiment with a PSO algorithm to solve a continuous optimisation problem. The lab emphasises how simple agent-based behaviours can collectively produce powerful optimisation performance.

Through hands-on coding and guided experimentation, students will explore the relationship between algorithm parameters, convergence behaviour, and solution quality.

Lab Learning Outcomes

By the end of this lab, you will be able to:

- Understand the principles of **swarm intelligence** and how they apply to optimisation problems
- Implement a working **Particle Swarm Optimisation algorithm**
- Analyse the effect of key PSO parameters (inertia, cognitive and social coefficients)
- Evaluate algorithm performance using objective functions and convergence metrics
- Develop intuition for exploration vs exploitation in population-based search

Run the docker container

Visit [Docker webpage](#) [\[/d2l/common/dialogs/quickLink/quickLink.d2l?ou=1118412&type=content&rcode=ntu-2841232\]](#) **to learn more about docker.**

If you are using your personal laptop - ensure that the steps described in [Install Docker Desktop on Windows machines](#) [\[/d2l/common/dialogs/quickLink/quickLink.d2l?ou=1118412&type=content&rcode=ntu-2841458\]](#) have been completed successfully.

1. Start docker desktop - **if you are using a lab PC**, ensure that your **DOCKER DESKTOP** is configured to use at least **4GB of RAM (Docker Desktop->Settings->Resources)**! **Ask your lab tutor for support if required!**
2. Start PowerShell (Windows) or Terminal (Linux/Mac OS) and run the following commands:

```
docker volume create docker_comp40771

docker pull pedrombmachado/nest:comp40771

docker run -d --rm --name comp40771_container -v docker_comp40771:/opt/data -e
NEST_CONTAINER_MODE=jupyterlab -p 8081:8080 pedrombmachado/nest:comp40771
```

Fork and clone the repository and run the Jupyter notebook

Forking and cloning the project from the git repository

Step 1: Login into <https://olympus.ntu.ac.uk/> [https://olympus.ntu.ac.uk/] using your NTU credentials.

Step 2: Goto the https://olympus.ntu.ac.uk/comp40731/comp40731_lab1 [https://olympus.ntu.ac.uk/comp40731/comp40731_lab1] and fork it

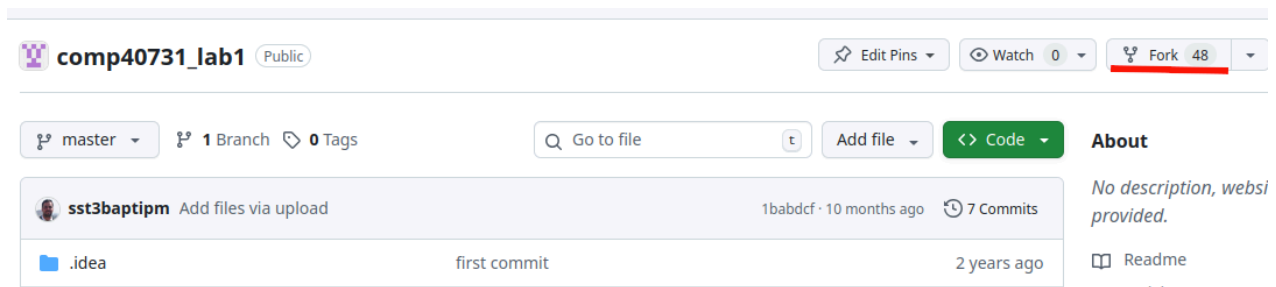


Figure 1: Forking a repository

Step 3: Goto your account (e.g. N123456/comp40771_lab1) hit the button **Code** and copy the repository url.

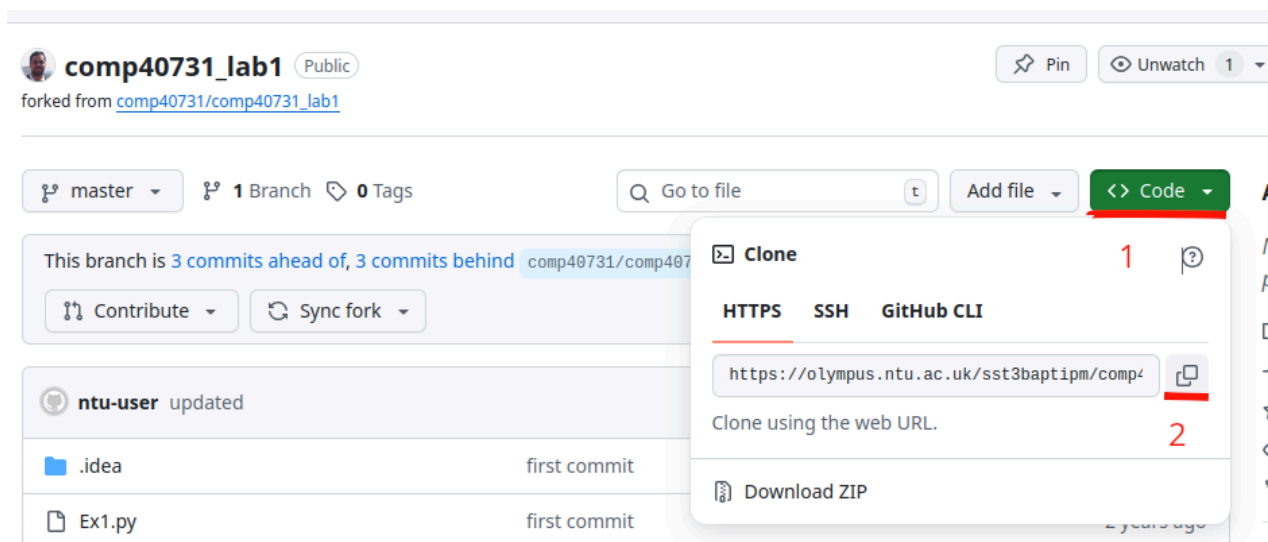


Figure 2: How to copy the repository url.

Step 4: Open a tab in your favourite browser (e.g. Chrome, Firefox, Edge, Safari, etc.), enter **http://localhost:8081/lab** and open a terminal.

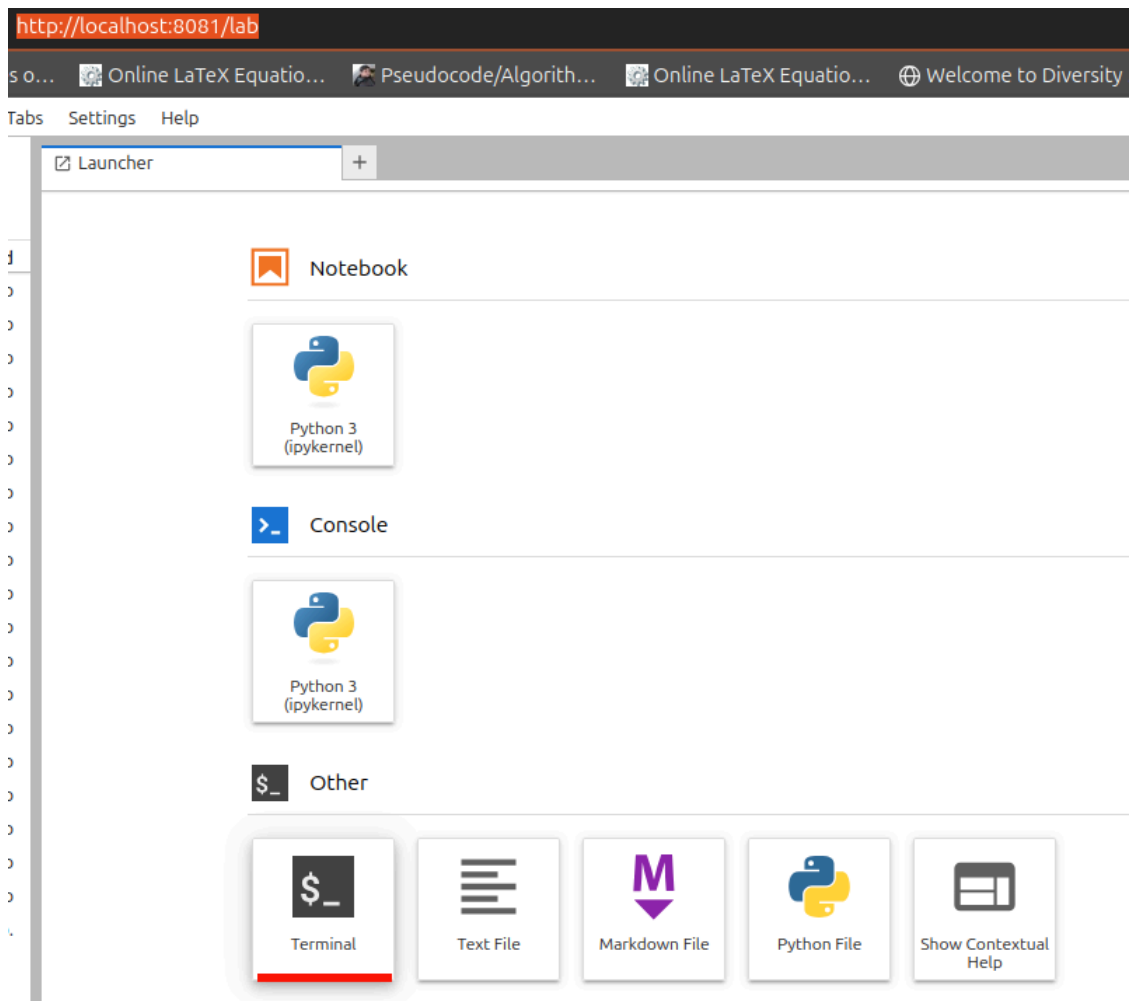
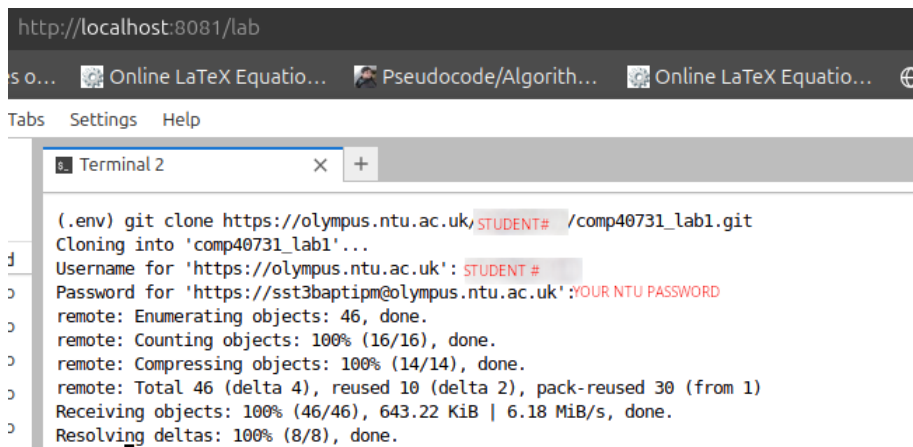


Figure 3: Open a terminal in the Jupyter Lab

Step 5: Clone the repository

and enter `git clone https://olympus.ntu.ac.uk/YOUR-STUDENT-NUMBER/comp40771_lab1`

where YOUR-STUDENT-NUMBER is your student number. Enter your student ID in the field Username and your NTU password in the field Password.



```
http://localhost:8081/lab
s o... Online LaTeX Equatio... Pseudocode/Algorith... Online LaTeX Equatio...
Tabs Settings Help

Terminal 2
(.env) git clone https://olympus.ntu.ac.uk/STUDENT# /comp40731_lab1.git
Cloning into 'comp40731_lab1'...
Username for 'https://olympus.ntu.ac.uk': STUDENT#
Password for 'https://sst3baptipm@olympus.ntu.ac.uk': YOUR NTU PASSWORD
remote: Enumerating objects: 46, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 46 (delta 4), reused 10 (delta 2), pack-reused 30 (from 1)
Receiving objects: 100% (46/46), 643.22 KiB | 6.18 MiB/s, done.
Resolving deltas: 100% (8/8), done.
```

Figure 4: Cloning the repository

Inspect, analyse and complete the lab activities

Step 6: Open the **comp40771_lab1** folder, followed by the **Lab1.ipynb** and **Restart the Kernel and run all cells** (see Figure 5).

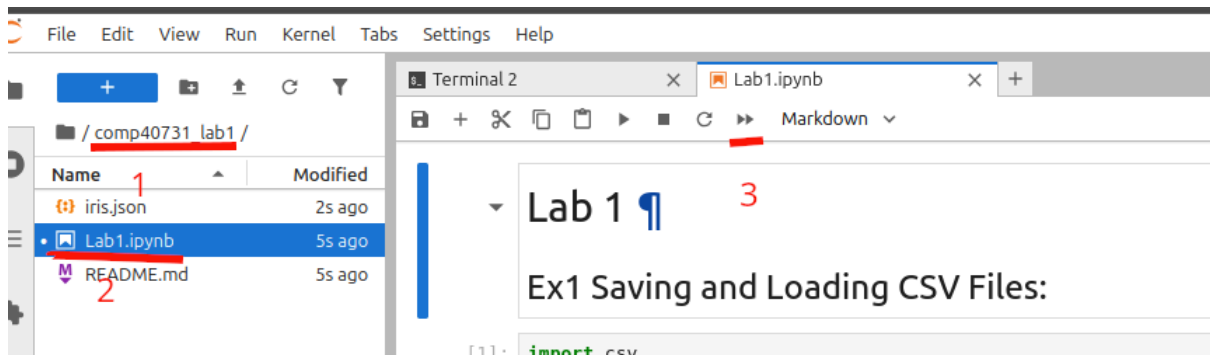


Figure 5: Opening the **comp40731_lab1** folder and **Lab1.ipynb** and **Restart the Kernel and run all cells**

Apply your knowledge

To deepen your understanding of how ANNs learn and perform, complete the following **two applied exercises**.

These tasks are designed to help you explore how architectural and training choices affect model performance and generalisation.

Task 1 - Explore the Effect of Network Architecture

Goal: Investigate how the **depth and width** of the neural network affect its ability to fit nonlinear data.

Steps:

1. Modify the ANN architecture by changing:
 - The number of hidden layers (e.g., 1, 2, 4, or 6 layers).
 - The number of neurons per layer (e.g., 16, 32, 64, 128).
 - The activation functions (e.g., relu, tanh, sigmoid).
2. Retrain the network on the same dataset.
3. Record and compare the **training loss**, **validation loss**, and **R²** score.
4. Plot all configurations together and discuss:
 - Which architectures **overfit** or **underfit**?
 - How does **network capacity** influence learning speed and final performance?

Hint:

Deeper or wider networks can model more complex patterns but may require **more training data** and **regularisation** (e.g., dropout, early stopping).

Task 2 - Test Generalisation and Robustness

Goal: Examine how the ANN handles **noisy or shifted data** compared to Linear Regression.

Steps:

1. Add **Gaussian noise** or **shift** the test data slightly (e.g., $X_{\text{test}}^{\text{new}} = X_{\text{test}} + \mathcal{N}(0, 0.2)$)).
2. Evaluate both the **ANN** and **Linear Regression** on the modified test set.
3. Compare the change in **MSE** and **R²** for both models.
4. Visualise how the ANN predictions behave under noise — are they more stable or more sensitive?

Hint:

Well-trained ANNs tend to generalise better to unseen or slightly distorted data, but if overfitted, they may perform worse than simpler models.

Task 3: Commit and push your source code

Open a terminal and navigate to the folder comp40731_lab1

```
cd /opt/data/comp40771_lab1
```

Add all the changes, commit the changes

```
git config --global user.name "Student Name"  
git config --global user.email "student@ntu.ac.uk"  
git add *  
git commit -m "update"
```

Push the changes to Olympus

```
git push
```

Stop the container

Run

```
docker stop comp40771_container
```

References

[1] Bhattacharyya, S. (2019). *Hybrid Computational Intelligence*. Taylor & Francis.

[2] Kumar, T.A.K.E.G.J.V.R.B.A.K. (2024). *Computational Intelligence*. Wiley Global Research (STMS).

[Go to top](#)
